Thank you for watching this N-Educate tutorial.

In this video, you will learn how to prepare your completed Google AI Studio Android app so GitHub Actions can automatically build the debug APK, publish it to GitHub Releases, and give you a direct APK download link you can open in your browser.

For further assistance, support, and access to more prompts, codes, and step-by-step resources, **join our Skool community using the link in the video description.**

Google AI Studio Prompt to Automatically Build and Download Android APK with GitHub Actions

---

Prompt Start Here
—----------
Prepare this completed Google AI Studio Android app for automatic debug APK building and direct APK downloading through GitHub Actions and GitHub Releases.

IMPORTANT WORKFLOW:

You must first create the GitHub Actions APK build workflow inside this project.

Do NOT require the GitHub repository to already exist.

After creating the workflow, give me a very short NEXT STEP instruction telling me to:

1. Publish/push this completed app to GitHub and create the repository.
2. Return to this same Gemini chat.
3. Paste ONLY the full GitHub repository name in this format:

username/repository-name

Example:

myusername/my-android-app

When I later send only a value matching:

username/repository-name

treat it as the GitHub repository identity for this project.

Do NOT edit the Android project again at that point.

Do NOT create another workflow.

Do NOT ask me for the GitHub URL.

Do NOT ask me for the owner and repository separately.

Use the full repository name I provide to generate this permanent direct APK download URL:

https://github.com/USERNAME/REPOSITORY/releases/latest/download/app-debug.apk

Then give me that completed direct APK download link clearly and tell me to allow the GitHub Actions APK build and GitHub Release process to finish before using the link.

---

1. Verify Android Gradle Plugin

Before creating anything, confirm that:

gradle/libs.versions.toml

contains exactly:

agp = "9.1.1"

If the AGP version is not exactly 9.1.1:

- Stop immediately.
- Report the detected AGP value.
- Do not create the workflow.
- Do not modify any file.

If the AGP version is exactly 9.1.1, continue.

---

2. Repository modification restriction

Create only:

.github/workflows/build-apk.yml

Do not create, edit, delete, rename, format, upgrade, or replace any other repository file.

Do not modify:

- Android source code
- App design
- Features
- Package name
- Application ID
- AndroidManifest.xml
- Resources
- Dependencies
- Gradle project files
- Signing configuration
- Project settings

The workflow must:

- Build the debug APK.
- Preserve the APK as a normal GitHub Actions artifact.
- Publish the same APK under GitHub Releases.
- Make the latest successful APK available through one permanent direct download URL.

---

3. Workflow triggers

Create a GitHub Actions workflow that:

- Runs automatically on every push to the main branch.
- Also supports workflow_dispatch.
- Does not run on pull requests.
- Uses ubuntu-latest.
- Checks out the complete repository.
- Runs all Gradle commands from the repository root.

This means that when I first publish/push this project to GitHub after you create the workflow, GitHub Actions should automatically begin building the APK.

---

4. Workflow permissions

Add only:

permissions:
  contents: write

Do not request unnecessary permissions.

Do not use:

- Personal access token
- Repository secret
- External API key
- Deployment key
- Custom credential

Use only the workflow's built-in GitHub token:

${{ github.token }}

---

5. Java setup

Set up Temurin Java 17 using:

actions/setup-java

Java must be configured before Gradle or any Gradle command.

Add a step named:

Verify Java environment

That step must:

- Run java -version.
- Print JAVA_HOME.
- Detect the active Java major version.
- Confirm the active Java version is 17 or newer.
- Fail clearly if Java is lower than 17.
- Fail clearly if Java cannot be detected.

---

6. Gradle setup

Set up exactly:

Gradle 9.3.1

using:

gradle/actions/setup-gradle

Do not use:

- Gradle 8.13
- Repository Gradle wrapper
- ./gradlew
- gradlew
- latest
- current
- release-candidate
- nightly
- Any Gradle version other than exactly 9.3.1

Run all Gradle commands from the repository root.

---

7. Temporary debug keystore

Before building, add a step named:

Create temporary debug keystore

That step must:

- Check whether ./debug.keystore already exists.
- Generate it only when it is missing.
- Use keytool from the configured Java installation.

Use exactly:

Keystore password:
android

Key alias:

androiddebugkey

Key password:
android

Distinguished name:
CN=Android Debug,O=Android,C=US

Algorithm:
RSA

Key size:
2048

Validity:
10000 days

After generation:

- Confirm ./debug.keystore exists.
- Fail clearly if it cannot be created.

Never:

- Commit the keystore.
- Cache the keystore.
- Print the keystore.
- Expose its contents.
- Upload the keystore as an artifact.
- Upload the keystore to GitHub Releases.
- Preserve the keystore outside the temporary GitHub Actions runner.

---

8. Build the debug APK

Build the debug APK using exactly:

gradle :app:assembleDebug --stacktrace --no-daemon

Do not replace this command with:

- ./gradlew
- gradlew

- assembleRelease
- bundleRelease
- assemble
- build
- Any other Gradle build command

---

9. Verify the APK

After the build, verify that this exact file exists:

app/build/outputs/apk/debug/app-debug.apk

If the exact file is missing:

- Search only inside app/build/outputs/apk.
- Print APK paths discovered there.
- Do not search the entire repository.
- Do not select a release APK.
- Do not select an Android test APK.
- Do not select an APK from another module.
- Do not upload or release an unrelated APK.
- Fail clearly.

Confirm that the APK:

- Exists.
- Is a regular file.
- Is not empty.
- Has the .apk extension.
- Is exactly:

app/build/outputs/apk/debug/app-debug.apk

---

10. Upload GitHub Actions backup artifact

After APK verification, add a step named:

Upload debug APK artifact

Use:

actions/upload-artifact

Upload only:

app/build/outputs/apk/debug/app-debug.apk

Name the Actions artifact exactly:

app-debug-apk

Configure artifact upload to fail if the APK is missing.

Do not upload:

- debug.keystore
- Credentials
- Source code
- Gradle caches
- Build caches
- Configuration files
- Secrets
- Logs containing sensitive information
- google-services.json
- Firebase configuration
- Signing files

The normal Actions artifact must remain available as a backup even after GitHub Release publishing is added.

---

11. Prepare APK release asset

After the Actions artifact is uploaded successfully, add a step named:

Prepare APK release asset

Create a temporary directory inside the GitHub Actions runner.

Do not create or commit this directory in the repository.

Copy only:

app/build/outputs/apk/debug/app-debug.apk

into the temporary directory.

The GitHub Release asset filename MUST ALWAYS be exactly:

app-debug.apk

Do not add:

- Repository name
- Workflow run number
- Workflow attempt
- Commit SHA
- Date
- Timestamp
- Version number
- Random characters

Every GitHub Release must contain its APK using the same asset filename:

app-debug.apk

Do not compress the APK.

Do not place the APK inside a ZIP.

Do not rename the original APK inside the repository.

Do not modify the original APK.

Export the prepared release asset path through GITHUB_OUTPUT.

---

12. Publish APK to GitHub Releases

After the normal Actions artifact is uploaded and the release asset is prepared, add a step named:

Publish APK to GitHub Releases

Use:

gh release create

Do not use an unverified third-party GitHub Release action.

Authenticate using:

GH_TOKEN: ${{ github.token }}

Before release creation:

- Run gh --version.
- Confirm GitHub CLI is available.
- Confirm the prepared APK exists.
- Confirm it is non-empty.
- Confirm it ends in .apk.
- Confirm the release asset filename is exactly app-debug.apk.
- Do not attempt release creation if APK validation fails.

---

13. Unique GitHub Release tag

Every workflow execution must receive a unique release tag using both:

GITHUB_RUN_NUMBER

and:

GITHUB_RUN_ATTEMPT

Use this tag format:

debug-apk-build-GITHUB_RUN_NUMBER-GITHUB_RUN_ATTEMPT

Example:

debug-apk-build-12-1

Construct the tag dynamically at workflow runtime.

Do not use a fixed tag such as:

- latest
- debug

- build
- v1
- apk

Do not overwrite, move, delete, or reuse an existing release tag.

The unique tag must prevent conflicts when:

- A new commit is pushed.
- The workflow is run manually.
- A failed workflow is rerun.
- Another run attempt occurs.

---

14. GitHub Release contents

Create a published GitHub Release.

Release title:

Debug APK Build #GITHUB_RUN_NUMBER

Release tag:

debug-apk-build-GITHUB_RUN_NUMBER-GITHUB_RUN_ATTEMPT

Target commit:

GITHUB_SHA

Release notes must clearly state:

- This is an automatically generated debug APK.
- The APK was built by GitHub Actions.
- GitHub Actions run number.
- GitHub Actions run attempt.
- Source commit SHA.
- APK is intended for testing.
- APK is not a Play Store production release.
- APK is signed only with a temporary debug keystore.

Attach only:

app-debug.apk

as the manually uploaded release asset.

Do not manually attach:

- ZIP files
- Source-code archives
- tar.gz files
- debug.keystore
- Credentials
- Configuration files
- Gradle files
- Build caches
- Logs
- Secrets
- Any unrelated APK

GitHub may automatically display its standard:

Source code (zip)

and:

Source code (tar.gz)

links.

Do not attempt to create, upload, replace, remove, or modify those platform-generated links.

Mark the newly created successful release explicitly as the latest release using:

--latest

Do not create the release as:

- Draft
- Prerelease
- Immutable release requiring later manual publication

The release Assets section must contain:

app-debug.apk

---

15. Permanent APK direct-download system

The GitHub Release asset filename must always remain:

app-debug.apk

Because the newest successful release is marked as latest, the permanent APK download URL format is:

https://github.com/USERNAME/REPOSITORY/releases/latest/download/app-debug.apk

Do NOT use:

- GitHub Actions artifact URLs
- Temporary URLs
- A specific release tag URL
- A run-number URL
- A release page as the primary download link

The permanent URL must always use:

/releases/latest/download/app-debug.apk

Do not attempt to determine USERNAME or REPOSITORY before the GitHub repository exists.

Do not invent the repository identity.

I will provide the full GitHub repository name after publishing the project.

---

16. Release failure protection

The GitHub Actions artifact must upload BEFORE release publishing begins.

Configure the release publishing step with:

continue-on-error: true

Assign an id to the release step so its outcome can be checked afterward.

If release publishing fails:

- Do not remove the successfully built APK.
- Do not remove the Actions artifact.
- Do not falsely report release success.
- Do not create a duplicate fallback release.
- Do not use another release action.
- Clearly report that the release failed.
- Clearly state that the Actions artifact named app-debug-apk remains available.

Release failure must not make a successful APK build appear to have failed if the APK and Actions artifact were successfully generated.

---

17. Report APK result

After the release attempt, add a step named:

Report APK download result

This step must always run after the release attempt.

If release publishing succeeded:

- Confirm the release using gh release view.
- Print the release tag.
- Print the release title.
- Print the release page URL.
- Print the APK filename:

app-debug.apk

- Construct the latest direct APK URL internally using:

https://github.com/${GITHUB_REPOSITORY}/releases/latest/download/app-debug.apk

- Add the direct APK URL to GITHUB_STEP_SUMMARY.
- Label it:

DIRECT APK DOWNLOAD

- State that the normal Actions artifact app-debug-apk remains available as a backup.

If release publishing failed:

- Print a clear warning.
- State that the APK build succeeded if it did.
- State that app-debug-apk remains available.
- Do not falsely claim the new APK is available through the direct latest-release link.

Authenticate gh commands using:

GH_TOKEN: ${{ github.token }}

---

18. Security restrictions

Do not create fake:

- google-services.json
- Firebase files
- API keys
- Credentials
- Personal access tokens
- Release signing files
- Production keystores
- Missing secrets
- Placeholder secrets

Do not expose:

- GITHUB_TOKEN
- GH_TOKEN
- Environment credentials
- Repository secrets
- Keystore contents

Do not echo authentication tokens into workflow logs.

Do not persist credentials beyond the temporary GitHub Actions runner.

Do not modify repository settings.

Do not enable GitHub Pages.

Do not create a website.

Do not create a download website.

Do not create a second workflow.

Do not commit an APK to the source repository.

Do not create a GitHub Release manually while preparing the project.

The release must be created only when the generated workflow runs after I publish/push the project to GitHub.

---

19. Final workflow validation

Before finishing, confirm:

- Only .github/workflows/build-apk.yml was created.
- No existing file was modified.
- AGP 9.1.1 was confirmed.
- Java 17 is configured before Gradle.
- Active Java version is verified.
- Exactly Gradle 9.3.1 is used.
- Repository Gradle wrapper is not used.
- Temporary debug keystore is created before building.
- Temporary keystore is never uploaded.
- Build command is exactly:

gradle :app:assembleDebug --stacktrace --no-daemon

- APK path is exactly:

app/build/outputs/apk/debug/app-debug.apk

- Actions artifact name is exactly:

app-debug-apk

- Actions artifact is uploaded before release publishing.
- Release asset is a real APK and not a ZIP.
- Release asset filename is exactly:

app-debug.apk

- Every workflow execution receives a unique release tag.
- GITHUB_RUN_NUMBER and GITHUB_RUN_ATTEMPT are used in the tag.
- Release targets GITHUB_SHA.
- New successful release is marked latest.
- Built-in GitHub token is used.
- contents: write permission is configured.
- No personal access token is needed.
- No repository secret is needed.
- Release failure leaves the Actions artifact available.
- No Android application/project file was changed.
- No APK was committed to the source repository.
- Permanent APK URL uses:

/releases/latest/download/app-debug.apk

---

20. FIRST RESPONSE — KEEP THIS VERY SHORT

After creating the workflow, do NOT give me a long explanation.

Give me only a short summary similar to:

APK BUILD WORKFLOW READY

AGP:
9.1.1

Java:
17+

Gradle:
9.3.1

APK:
app/build/outputs/apk/debug/app-debug.apk

Release APK:
app-debug.apk

Then display:

NEXT STEP

Publish/push this app to GitHub now.

This first GitHub push will also trigger the automatic APK build because the workflow is already included in the project.

After GitHub creates the repository, return to this Gemini chat and paste ONLY the full repository name.

Use this format:

username/repository-name

Example:

myusername/my-android-app

Do not paste anything else.

I will generate your permanent direct APK download link.

Do NOT try to detect the repository automatically.

Do NOT invent a repository name.

Wait for me to provide it.

---

21. FOLLOW-UP BEHAVIOR AFTER I PROVIDE THE REPOSITORY NAME

If my next message contains only a repository identity in this format:

username/repository-name

example:

appsteck/daily-quote-apk

do the following:

1. Treat the first part as the GitHub owner/username.
2. Treat the second part as the GitHub repository name.
3. Do not modify any project file.
4. Do not create another workflow.

5. Do not rebuild the Android app.
6. Do not ask additional questions.
7. Do not attempt to publish anything.
8. Construct this URL using the exact repository name I provided:

https://github.com/USERNAME/REPOSITORY/releases/latest/download/app-debug.apk

Then respond very briefly using this format:

==========

📱 DIRECT APK DOWNLOAD

https://github.com/USERNAME/REPOSITORY/releases/latest/download/app-debug.apk

Your GitHub Actions APK build started when you published/pushed the project.

Allow approximately 7–9 minutes for the build and GitHub Release process to finish.

If the link is not ready yet, wait a little longer and try again.

Once the release finishes, paste this link into Chrome to download app-debug.apk directly.

==========

Do not add unnecessary instructions.

Do not send me to the GitHub Actions page unless the direct APK link still does not work after the build has had enough time to complete.

Do not change the URL format.

Do not use a release tag in the URL.

Do not replace app-debug.apk with another filename.

Do not publish, commit, sync, push, tag, or manually create a GitHub Release now.

Create only the workflow file and wait for me to manually publish/push the project to GitHub.

Prompt End

—----------